



Tips☆☆° NO GLYPH ☆°☆☆ NO GLYPH

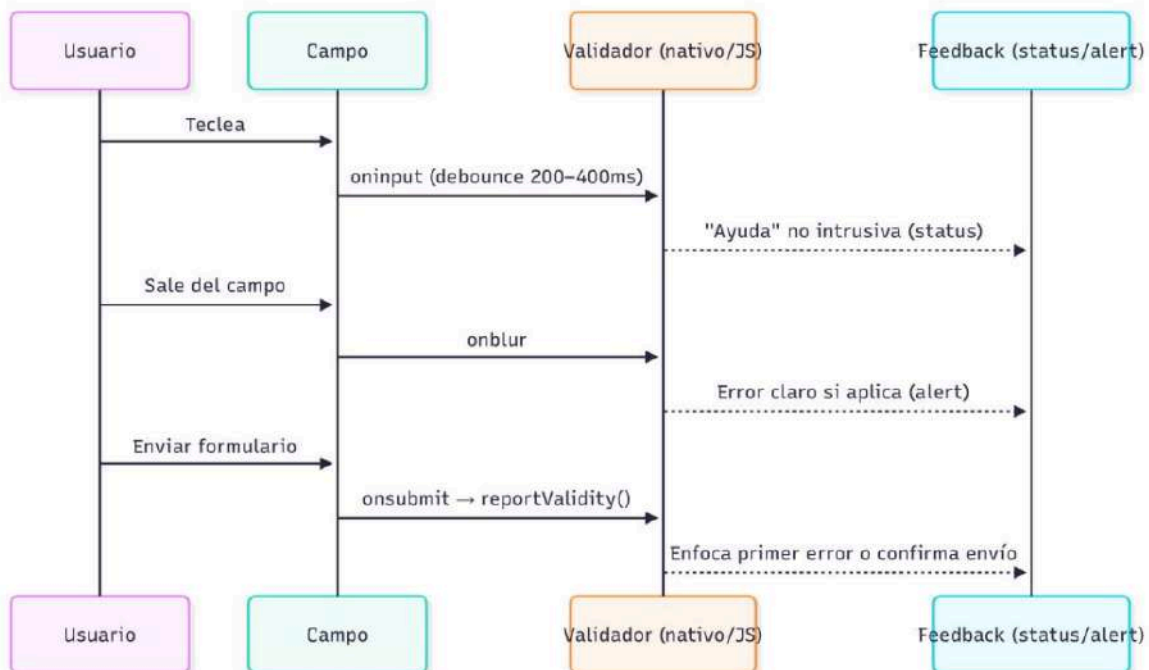
📅 fecha👤👤	31 de marzo de 2026
📄 resumen IA👤👤👤👤	No content

Utilizar eventos de JavaScript para hacer validaciones de input en tiempo real, una vez que termine de escribir no validar por cada tecla. Tomar cuenta lo siguiente:

No interrumpas a cada tecla. Usa una estrategia combinada:

- onblur (al salir del campo): formato/obligatoriedad.
- oninput con debounce (200–400 ms): contadores o "fuerza" de contraseña.
- onsubmit: última barrera; enfoca el primer error.

Para input normales (nombre, apellidos, dirección) emplear las validaciones nativas, se puede poner validación en tiempo real con campos como cédula



Hacer un evento para desplegar del menú horizontal un submenú como en la segunda ventana de INEC

Hacer que el usuario teclee menos emplear autocompletado con autocomplete (Es una buena práctica ¿? la ingeniera probará en mi computadora por lo que no se si seria bueno esto)

Añadir atributos aria

Bajar la carga cognitiva en formularios

Cuando hay muchos campos, dividir en pasos baja la carga cognitiva. Cada paso debe tener

título (<h2>), progreso (<progress value="1" max="3">) y gestión del foco (al avanzar, el foco va al título del nuevo paso). No borres datos al volver.

Internacionalización

Poner el lenguaje en el head y poner UTF-8

Fechas/números: la UI puede mostrarse local, pero guarda en ISO/UTC.

- Mensajes: en el idioma del documento (<html lang="es">), sin tecnicismos.
- Seguridad: el cliente mejora la experiencia; el servidor decide. Revalida y sanea, usa

HTTPS, protege CSRF si hay sesión, y verifica tiempo real y tamaño en archivos (no confíes solo en accept).

Hacer que el lector de la página cumpla con la estructura del DOM

No usar los h1-6 por estética, si se quiere cambiar la letra por estética usar CSS

Debe existir un único h1, puede existir varios h2 que deben estar dentro del h1, si pongo h3 debe estar dentro del h2

En imagenes debo usar porcentaje, para lo demás puedo usar em o rem

No es muy sugerido usar tablas debido a la adaptabilidad. Emplear flexbox en lugar de tablas

Procurar poner id o clases para poner los estilos



Clase CSSW

📅 fecha	27 de marzo de 2026
📄 resumen IA	No content

Debemos hacer que se base en la 3WC y WCAG.

Demos usar lo nativo de HTML5 para que interprete del navegador lo lea más rápido

Nombrar todo con label

Enlazar ayudas/errores con aria-describedby

Dejar que el navegador ya valide lo básico

Formularios

- Una tarea por formulario
- Primero campos obligatorios luego campos opcionales
- Nombre accesible con label for
- Ayuda/errores junto al campo y vincularlos
- Botón principal único (type="submit")

Desde el punto de vista de seguridad, rendimiento y optimización es mejor manejar todo por objetos. Al servidor envío un objeto, el servidor envía un JSON que el cliente debe procesar

Tipos de Entrada:

Usar validaciones, atributos aria, todos los tipos de input

Fieldset es un objeto para agrupar objetos del mismo tipo

Tipo	¿Para qué sirve?	Ventaja práctica de UX/A11y
email	Correos	Formato básico + teclado móvil adecuado
url	Enlaces	Valida esquema; reduce errores
tel	Teléfonos	Teclado numérico (móvil)
number	Cantidades exactas	min/max/step; evita letras
range	Preferencias con deslizador	Añade un valor visible con <output>
date/time	Fechas/horas	Selectores nativos; menos JS
file	Subir archivos	accept, multiple; valida también en servidor
color	Selector de color	Personalización nativa

span y small para etiquetas conectadas con aria-describedby

pattern para restringir la extensión del campo y el inputmode poner para que tipo de dato manejará el campo

la validación se debe hacer en las tres capas

Validar en tiempo real, es decir al pasar al siguiente campo que se verifique si el anterior esta bien (JavaScript)

Algoritmo para verificar que la cedula sea correcta, poner mensajes claros

El JavaScript permite modificar al DOM(elementos del HTML)

JAVASCRIPT

- Cada variante es un objeto, se lo suele guardar como constante
- Cada objeto tiene atributos (modificar CSS) y eventos
- Puede modificar los elementos del DOM

Autocompletado

autocomplete permite rellenar con los datos guardados en el perfil del navegador

Bajar la carga cognitiva en formularios

Cuando hay muchos campos, dividir en pasos baja la carga cognitiva. Cada paso debe tener

título (<h2>), progreso (<progress value="1" max="3">) y gestión del foco (al

avanzar, el foco va al título del nuevo paso). No borres datos al volver.

Validar solo lo que no hace HTML

Al emplear JAVASCRIPT validar lo que no valida por si solo el html

Crea el formulario con validaciones nativas y las que falten crealas en JavaScript

Internacionalización

Poner el lenguaje en el head y poner UTF-8

Fechas/números: la UI puede mostrarse local, pero guarda en ISO/UTC.

- Mensajes: en el idioma del documento (<html lang="es">), sin tecnicismos.
- Seguridad: el cliente mejora la experiencia; el servidor decide. Revalida y sanea, usa

HTTPS, protege CSRF si hay sesión, y verifica tiempo real y tamaño en archivos (no confíes solo en accept).

HTTPS encripta el código para que si lo capturan no sepan que dice

Si no se sigue el orden semántico la página se demorará en renderizarse

Media Query es adaptar y cambiar la **presentación visual** de tu página dependiendo de las características del dispositivo en el que se está viendo

Encabezados: el "índice" del documento

HTML nombra y ordena, CSS acomoda y JS anima

No usar los h1-6 por estética, si se quiere cambiar la letra por estética usar CSS

Debe existir un único h1, puede existir varios h2 que deben estar dentro del h1, si pongo h3 debe estar dentro del h2

Semántica fina: "qué es" cada pieza

Si se quema el estilo en el html, es este quien gana prioridad

- Imágenes con contexto: figure + figcaption. El alt describe la información visual; el pie añade contexto.
 - Fechas: time datetime="YYYY-MM-DD" (texto humano + valor legible por máquinas).
 - Abreviaturas: abbr title="..." (p. ej., WCAG).
 - Código/teclado/salida: code, kbd, samp, y pre para formato preservado.
 - Énfasis/Importancia: em y strong (significado, no solo estilo) (em toma la medida que tiene por defecto el navegador, em toma la medida que tiene el estilo) (En imágenes debo usar porcentaje).
 - Resaltado contextual: mark (p. ej., término buscado) (resalta el término buscado).
 - Definiciones/Pares clave-valor: dl + dt + dd.
- Los datos deben leerse de un archivo JSON, los datos de clientes, productos, proveedores, facturas deben cargarse desde JSON (No parar bola a lo que dice estáticos). Por lo que debemos hacer que las estructuras se creen de forma dinámica en base a los datos
 - Sección colapsable: details + summary (acordeón nativo y accesible).

No es muy sugerido usar tablas debido a la adaptabilidad

Procurar poner id o clases para poner los estilos

2.2.6. ARIA: cuándo sí y cuándo no

ARIA no reemplaza HTML. Primero elige el elemento correcto; luego, si no existe equivalente

nativo, usa ARIA para completar:

- Varias nav → diferéncialas con aria-label="Principal", aria-label="Breadcrumbs".
- Acordeón custom (si no usas details): role="button", aria-expanded, aria-controls, y teclas Enter/Espacio.
- Evita roles redundantes: no pongas role="navigation" en nav, ni role="main" en main.

ARIA es para complementar a HTML no para sustituirlo

2.2.7. Anti-patrones comunes (y su arreglo)

- "Div soup" (todo con div) → Usa landmarks y semántica fina.
- Encabezados por tamaño (usar h4 "porque es más chico") → Usa el nivel correcto y

estiliza con CSS.

- section sin encabezado → Añade h2/h3 dentro o usa div.
- Figuras sin figcaption cuando la imagen requiere contexto → Añádalo.
- Tablas para layout → Maqueta con Flexbox/Grid (Semana 3).
- Roles ARIA arbitrarios que contradicen HTML → Valida y elimina lo redundante

Utilizar eventos de JavaScript para hacer validaciones de input en tiempo real, una vez que termine de escribir no validar por cada tecla. Tomar cuenta lo siguiente:

No interrumpas a cada tecla. Usa una estrategia combinada:

- onblur (al salir del campo): formato/obligatoriedad.
- oninput con debounce (200–400 ms): contadores o "fuerza" de contraseña.
- onsubmit: última barrera; enfoca el primer error.

Para input normales (nombre, apellidos, dirección) emplear las validaciones nativas, se puede poner validación en tiempo real con campos como cédula

Las páginas web deben cumplir con el estándar AA

Las WCAG organizan sus pautas en cuatro principios que recordamos como POUR: Perceptible, Operable, Comprensible y Robusto.

Principio	Acción práctica (AA)	Cómo probar rápido
Perceptible	alt útil en imágenes informativas; figure+figcaption si hace falta contexto	Navega con SR (NVDA/VoiceOver); inspecciona alt
Perceptible	Contraste $\geq 4.5:1$ (texto normal) y $\geq 3:1$ (texto grande)	Lighthouse/Axe; comprobador de contraste
Perceptible	Reflow con zoom 200–400% sin scroll horizontal general	Ctrl/Cmd +; revisa columnas y tarjetas
Operable	Todo con teclado: Tab/Shift+Tab/Enter/Espacio	Recorre la página sin mouse
Operable	Foco visible y orden lógico	:focus-visible; prueba Tab en toda la UI
Operable	Modales accesibles (foco atrapado legit., aria-modal, cerrar con Esc)	Abrir/cerrar con teclado y lector
Comprensible	Etiquetas reales + aria-describedby para ayudas/errores	SR anuncia propósito + error
Comprensible	Jerarquía de encabezados (h1→h2→h3...) y navegación coherente	Vista de encabezados; inspección semántica
Robusto	Semántica nativa primero; ARIA solo si falta equivalente	Validador HTML; roles correctos en DevTools

2.4.2. Perceptible: que la información "llegue"

Imágenes. Si la imagen comunica algo, pon alt descriptivo; si es decorativa, alt="". Si requiere contexto, usa figure + figcaption.

2.4.3. Operable: teclado first

El sitio debe poder usarse completamente con teclado. Asegura foco visible y orden lógico. Modales accesibles (idea mínima). Deben: 1) atrapar foco, 2) anunciarse, 3) cerrarse con Esc, 4) devolver el foco al disparador.

2.4.4. Comprensible: que se entienda lo que pides

Usa lenguaje claro y etiquetas reales. Las ayudas y errores deben estar vinculados y anunciarse. Mantén jerarquía de encabezados (h1→h2→h3...) y patrones de navegación consistentes entre páginas (mismo lugar para menú, buscador, etc.).

2.4.5. Robusto: que funcione "a prueba de futuro"

Semántica nativa primero: header, nav, main, section, article, aside, footer, figure/figcaption, time, dl/dt/dd, details/summary.

ARIA solo cuando falte semántica nativa o para desambiguar (ej.: varias nav con aria-label).

Valida HTML/CSS, prueba en más de un navegador y, si puedes, con un lector

(NVDA en Windows, VoiceOver en macOS). Evita roles redundantes (role="navigation" en nav) o incorrectos (pueden confundir a las ayudas técnicas).

2.4.6. Anti-patrones frecuentes

- Quitar el foco por estética → Personalízalo, no lo elimines.
- Color como única señal → Acompáñalo con texto/ícono.
- Placeholders como etiquetas → Usa label for.
- Tablas para maquetar → Usa Flexbox/Grid (Semana 3).
- Componentes custom sin roles/teclas → Usa nativos o implementa roles y teclado completos.
- Modales que no devuelven el foco → Devuélvelo al disparador.

Emplear flexblox en lugar de tablas



Clase teórica 1 ★ ★ °. NO GLYPH ★. ° ☆ NO GLYPH

📅 fecha °. °	17 de marzo de 2026
📄 resumen IA . NO GLYPH + NO GLYPH NO GLYPH	No content

Escalabilidad: Poder adaptar nuevos procesos, servicios, sin afectar a la página web.

Seguridad:

Accesibilidad:

Rendimiento, seguridad y accesibilidad.

CSS para que la página se vea bien en un celular, laptop, computadora, televisión

Vamos a trabajar con

CI/CD Continuous Integration/Continuous Deployment

Dev/OPS Development Operaciones

Dev/Sec/Ops Development Security Operations

¿Cómo funciona el CI/CD, Dev/Sec/Ops?

¿Cómo a aplicarlo a GitHub?

Web

Escritorio a través de lenguajes de programación que tienen una interfaz y tienen dependencia del SO

Hibrida

Arquitectura cliente-servidor moderna

BDD → Internet/intranet (https) ← Logica-APIS/Api UI, Api Data → > UI

La información de la BDD es transformada por la API de Data en formato JSON, la pasa a la API UI, la UI toma esa información renderiza el json y la muestra.

El cliente solicita algo, ese mensaje llega a la intranet o internet para ir a la API UI, esta habla con la lógica para pedir a la BDD los datos, entonces la API data se comunica con la BDD o directamente la lógica se comunica con la BDD, entonces la BDD devuelve la información solicitada a la api de data o lógica y luego se transfiere a la api UI la cuál envía la información a la interfaz

La lógica se dispone a la API a través de la URL

La única forma de que la lógica se conecte con el front es la API o el protocolo http(s)

- Base de datos distribuidas, bdd local físico, bdd online
- Base de datos distribuidas: Distribuir en fragmentos
- SE DEBE MANEJAR EN TODO TRABAJO EL CLEINTE SERVIDOR MODERNO

El balanceador de carga es quien distribuye las solicitudes entre múltiples servidores

Los servidores de aplicaciones se encargan de ejecutar la lógica de negocio

Memoria caché se divide:

- BDD (vistas)
- Lógica (memorias temporales, paginación)
- Interfaz (guarda la información en el cliente)

Se aplica la seguridad en las tres capas: BD, lógica e interfaz

Las páginas web deben estar disponibles en 3 segundos

WebSockets se usa para aplicaciones en tiempo real

Seguridad en APIs

- HTTPS: protege la comunicación mediante cifrado TLS/SSL
- AUTENTICACIÓN Y AUTORIZACIÓN: gestión de identidad mediante OAuth 2.0
- Tokens JWT: mantiene sesiones sin necesidad de almacenar credenciales en el servidor
- CORS: controla que dominios pueden consumir los recursos y que permisos tiene.

CI/CD (trabajo colaborativo DEV/SEC/OPS) se trabajará con VS Code

DEV/SEC/OPS: Aplicación de la ingeniería de Software para trabajos colaborativos

El GitHub ya versiona

Investigar a profundidad el 1.4.2, implementando CI/CD, DEV/SEC/OPS y su aplicación en github

Consultar cómo con el GitHub se va hacer lo que dice en 1.4.2 aplicando CI/CD, DEV/SEC/OPS, hacer una presentación o documento

Entornos virtuales

Kubernetes

Consejos

- Al hacer proyectos hacer las versiones más recientes para que mi sistema tenga un tiempo de vida amplio
-

HTML

- Head
 - No se visualiza en el navegador
 - metadata (utf-8) /titulo
 - Style/ Viewport / keywords(permite SEO para que los buscadores busquen las palabras claves)
- Body
 - Si se visualiza en el navegador
 - DOM: elementos html que permiten la visualización en el navegador

navigation: corresponde a la navegación

aria-label: es una etiqueta

aria-live: se actualizará dinámicamente

Herramientas de validación y auditoría

El menú nav esta dentro de la cabecera por buena practia

figure me permite poner varias imágenes

figcaption permite colocar lista de textos o imagenes si la principal no aparece

DOM: construye un árbol que optimiza el tiempo y recursos. Contenido de HTML, son las etiquetas ubicadas jerarquicamente

Hosting

Carpeta física donde se guarda el directorio raíz de una página web

Las imagenes, música, etc. Deben estar dentro del direcvtorio raíz

Usualmente siempre hay un documento que se llama Index.html

Un directorio raíz por cada proyecto

Para salir un directorio atrás se pone ../

Son path virtuales o relativos al directorio raíz src='.././imagenes/ejemplo1.jpg'

No se debe poner path absoluto C:

<meta charset="UTF-8"> Cómo interpretar el texto

<meta name="viewport" content="width=device-width, Para que se ajuste a diferentes pantallas

Las páginas web se interpretan del lado del cliente

El cliente pide una página web, el servidor web envía el archivo y es el navegador quien interpreta el código

Estructura jerárquica del contenido

h1-h6

link: protocolo//dominio o servidor:puerto

Inicializa Git con npm

Subir los programas a Git

```
winget uninstall -i Git
```

¿Qué son los atributos aria?

Usado para accesibilidad, permite que un usuario pueda navegar con mi página web sin usar el mouse solo enter.

Tips

- Una página web sin elementos arias no es accesible
- Para accesibilidad hay que usar medidas relativas no medidas en pixeles, los cuales son em, rem, %. Se usa em, erm para letras, y % para imagenes
- Una clase me permite dar solo un formato para toda la clase en lugar de dar estilo a cada id
- Todas las páginas web deben tener accesibilidad
- Las tablas no son adaptables por lo que se recomienda no implementarlas